

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO MÔN:

AN TOÀN VÀ BẢO MẬT HỆ THỐNG THÔNG TIN

ĐỀ TÀI:

**Tối thiểu hóa ảnh hưởng: Một khung thuật toán bền vững để xử lý tính
non-submodular của hàm mục tiêu**

Giảng viên hướng dẫn: PGS.TS Đỗ Xuân Chợt

Nhóm: Lớp D23CQCT01 - Nhóm 06

Thành viên:

TRẦN VĂN KHÁNH	B23DCCN433
PHAN VĂN KHÔI	B23DCCN447
BÙI VĂN LÂM	B23DCCN477
DƯƠNG TRỌNG THẮNG	B23DCCN752

Hà Nội, tháng 5 năm 2026

Tối thiểu hóa ảnh hưởng: Một khung thuật toán bền vững để xử lý tính non-submodular của hàm mục tiêu

Trần Văn Khánh

Phan Văn Khôi

Bùi Văn Lâm

Dương Trọng Thắng*

Ngày 24 tháng 5 năm 2026

Tóm tắt nội dung

...
Từ khóa:...

1 Giới thiệu

Trong kỷ nguyên của các cơ sở hạ tầng kỹ thuật số siêu kết nối, An toàn và Bảo mật Hệ thống Thông tin đã trở thành một trong những mối quan tâm cấp thiết nhất đối với các tổ chức. Các mạng lưới hiện đại, từ mạng nội bộ doanh nghiệp cho đến các hệ sinh thái IoT lớn, tạo điều kiện thuận lợi cho việc trao đổi dữ liệu nhưng đồng thời cũng tiềm ẩn những lỗ hổng bảo mật nghiêm trọng. Lợi dụng cấu trúc của các mạng lưới này, các mối đe dọa không gian mạng như mã độc, mã độc tống tiền và các chiến dịch lừa đảo phishing có thể phát tán nhanh chóng thông qua các hiệu ứng lan truyền, dẫn đến việc thỏa hiệp hệ thống trên diện rộng và rò rỉ dữ liệu. Khi một tập hợp các nguồn độc hại (ví dụ: các máy tính đã bị nhiễm mã độc) bị phát hiện, các quản trị viên hệ thống phải lập tức triển khai các chiến lược ứng phó sự cố để ngăn chặn sự di chuyển của cuộc tấn công. Trong số các chiến lược đó, cách ly nút - bao gồm việc vá lỗ hổng, ngắt kết nối các máy chủ cụ thể, hoặc đình chỉ các tài khoản bị xâm nhập - là một biện pháp mang lại hiệu quả cao. Tuy nhiên, do giới hạn về nguồn lực vận hành và ngân sách bảo mật eo hẹp, việc vá hay cách ly toàn bộ các thực thể dễ bị tổn thương cùng một lúc là điều bất khả thi. Thách thức thực tiễn này đặt ra bài toán Giảm thiểu Lan truyền (Influence Minimization - IMIN) thông qua việc chặn nút, với mục tiêu tìm ra một tập hợp tối đa k nút trọng yếu cần chặn sao cho số lượng nút được bảo vệ khỏi sự lây lan mã độc (kỳ vọng) là lớn nhất.

Bài toán IMIN mang bản chất cực kỳ phức tạp về mặt tính toán, đã được chứng minh là NP-khó (NP-hard) và APX-khó (APX-hard) [?]. Các nghiên cứu ban đầu như của Wang và cộng sự [?] đã giải quyết bài toán IMIN trong mô hình Lan truyền Độc lập (Independent Cascade - IC) bằng cách sử dụng mô phỏng Monte-Carlo kết hợp với thuật toán greedy. Mặc dù vậy, phương pháp này đòi hỏi chi phí tính toán khổng lồ, khiến nó không khả thi khi triển khai cho các bài toán an toàn thông tin trên các mạng lưới quy mô lớn, nơi mà thời gian phản ứng nhanh là yếu tố sống còn. Gần đây, Xie và cộng sự [?] đã đề xuất một phương pháp ước lượng dựa trên cây dominator kết hợp với thuật toán tham lam cải tiến. Dù mang lại hiệu suất tốt hơn, phương pháp này vẫn yêu cầu tính toán lại hàm mục tiêu một cách đắt đỏ sau mỗi lần chọn một nút chặn. Đáng chú ý hơn, rào cản nền tảng của bài toán IMIN nằm ở chỗ hàm mục tiêu của nó không có tính submodular [?]. Do đó, việc áp dụng trực tiếp thuật toán tham lam truyền thống không thể cung cấp tỷ lệ xấp xỉ đảm bảo $(1 - 1/e)$ quen thuộc [?], và các phân tích lý thuyết dựa trên độ cong hay tỷ lệ submodularity chỉ tạo ra các giới hạn tầm thường và phụ thuộc nhiều vào dữ liệu.

Để vượt qua rào cản non-submodular, các nghiên cứu gần đây đã khám phá Khung xấp xỉ Sandwich (ví dụ: SandIMIN [?]), bằng cách kẹp hàm mục tiêu ở giữa một hàm cận dưới và một hàm cận trên, trong đó cả hai hàm cận đều có tính chất submodular. Dù cung cấp một bước đột phá về mặt lý thuyết, việc triển khai thực tế của SandIMIN lại gặp phải ba nhược điểm lớn. Thứ nhất, việc ước lượng các hàm cận đòi hỏi các cấu trúc lấy mẫu cực kỳ phức tạp và nặng nề về tính toán (cụ thể là chuỗi CP và tập LRR), đi lệch đáng

*Mã nguồn: <https://github.com/VanLam05/ATBM.git>

kể so với sự tối giản và tốc độ của phương pháp Lấy mẫu Ảnh hưởng Ngược (Reverse Influence Sampling - RIS) tiêu chuẩn. Thứ hai, khung Sandwich bắt buộc hệ thống phải thực thi ba thuật toán độc lập (tối đa hóa cận dưới, tối đa hóa cận trên, và một thuật toán heuristic ở giữa) để chọn ra kết quả tốt nhất. Điều này dẫn đến sự lãng phí tài nguyên vô ích và làm chậm trễ đáng kể thời gian ứng phó sự cố bảo mật. Thứ ba, tỷ lệ xấp xỉ lý thuyết của phương pháp này chỉ được hiển thị từ hàm chặn trên, là một hàm chặn khá lỏng, khiến kết quả về tỷ lệ xấp xỉ lý thuyết không được tốt.

Trong bài báo này, chúng tôi đề xuất một khung giải pháp mới, được tinh giản tối đa cho bài toán IMIN nhằm khắc phục trực tiếp các hạn chế của những nghiên cứu trước đó. Thay vì phụ thuộc vào khung Sandwich rườm rà, chúng tôi thiết kế toán học một hàm chặn trên chặt chẽ và tuân thủ nghiêm ngặt tính chất submodular cho hàm mục tiêu của IMIN. Để tối đa hóa hàm cận này một cách hiệu quả, chúng tôi đã điều chỉnh thuật toán tiên tiến nhất OPIM-C [?] - vốn được thiết kế cho bài toán Cực đại hóa Lan truyền (Influence Maximization - IMAX) - sang lĩnh vực IMIN. Khác biệt với SandIMIN, khung giải pháp của chúng tôi giới thiệu một quy trình lấy mẫu dựa trên đường đi được đơn giản hóa đáng kể, giúp duy trì độ phức tạp thời gian gần như tối ưu (gần tuyến tính) vốn có của các kỹ thuật RIS hiện đại. Bằng cách chỉ tập trung tối ưu hóa duy nhất một hàm cận trên chặt chẽ, cách tiếp cận của chúng tôi loại bỏ hoàn toàn yêu cầu phải chạy các thuật toán dư thừa, từ đó mang lại thời gian thực thi nhanh hơn vượt trội. Hơn nữa, công thức hóa tối giản cho phép chúng tôi chứng minh được một tỷ lệ xấp xỉ tốt hơn, đảm bảo sự chắc chắn về mặt lý thuyết cho các hoạt động ứng cứu không gian mạng.

2 Kiến thức cơ sở

Trong phần này, chúng tôi định nghĩa bài toán IMIN, sau đó tổng quan về các đề xuất hiện có và các hạn chế cốt lõi của chúng.

2.1 Định nghĩa bài toán

Xét một mạng lưới hệ thống thông tin được biểu diễn dưới dạng đồ thị có hướng $G = (V, E)$, trong đó V là tập hợp các nút với $|V| = n$, và E là tập các cạnh có hướng với $|E| = m$. Với mỗi cạnh $\langle u, v \rangle \in E$, u được gọi là hàng xóm đầu vào của v . Mỗi cạnh $\langle u, v \rangle$ được gán một xác suất lây lan $p(u, v) \in [0, 1]$, thể hiện xác suất nút u có thể truyền mã độc (hoặc thỏa hiệp) thành công nút v .

Mô hình lan truyền: Trong mô hình này, mỗi cung $(u, v) \in E$ có một xác suất ảnh hưởng là $p(u, v) \in [0, 1]$ (hay viết đơn giản là p_{uv}). Trước khi đi vào định nghĩa mô hình IC, ta quy ước $S_{-1} = \emptyset$. Xét đồ thị $G(V, E)$, bộ trọng số xác suất $p(\cdot)$ cho các cung và tập seed $S = S_0$. Các tập active S_t được sinh theo quy tắc sau:

1. Tại mỗi bước thời gian $t \geq 1$, tập S_t ban được gán là S_{t-1} .
2. Tiếp đó, với mỗi nút $v \notin S_{t-1}$, ta xét từng nút $u \in N^{\text{in}}(v) \cap (S_{t-1} \setminus S_{t-2})$ để thực hiện một phép thử kích hoạt tuân theo phép thử Bern(p_{uv}). Nếu thành công, ta thêm v vào S_t và nói rằng u kích hoạt v tại thời điểm t . Chú ý nếu nhiều nút cùng kích hoạt thành công v tại cùng thời điểm thì quy tắc vẫn vậy.

Thông thường, khi nút u mới được kích hoạt tại thời điểm $t - 1$, nó chỉ có một cơ hội duy nhất thử kích hoạt từng nút out-neighbor và các sự kích hoạt như vậy độc lập với nhau. Một khi một nút được kích hoạt, nó giữ nguyên trạng thái ấy. Như đã đề cập, tại thời điểm nào đó, không có thêm nút nào được kích hoạt, tức là $S_t = S_{t-1}$, quá trình lan truyền đang xét kết thúc với tập final active S_t . Một lần mô phỏng lan truyền từ S_0 như vậy còn được gọi là một *realization*. Gọi $\mathbb{E}[I_G(S)]$ là kỳ vọng số lượng nút bị lây nhiễm cuối cùng, quá trình tính $\mathbb{E}[I_G(S)]$ có thể tiếp cận được từ S trên toàn bộ không gian mẫu các realizations Ω .

Trong bài này, chúng tôi tập trung vào mô hình IC, vốn được sử dụng rộng rãi để mô phỏng sự phát tán thông tin và mã độc [?]. Đặc biệt, chúng tôi thiết lập xác suất lây lan tuân theo mô hình *Weighted Cascade (WC)*. Cụ thể, xác suất trên mỗi cạnh được định nghĩa là $p(u, v) = 1/|N_{\text{in}}(v)|$, với $N_{\text{in}}(v)$ là tập các nút có cạnh hướng tới v .

Chiến lược chặn nút: Khi một nút u bị chặn (được cách ly hoặc vá lỗi), xác suất của mọi cạnh trở tới u sẽ bị gán bằng 0. Khi đó, một số nút trước đây vốn có thể bị lây nhiễm từ S giờ có thể được cứu. Cho một

tập seed S và một tập block $B \subseteq (V \setminus S)$, gọi $D_S(B) = \mathbb{E}[I_G(S)] - \mathbb{E}[I_{G[V \setminus B]}(S)]$ là kỳ vọng số lượng nút được bảo vệ sau khi chặn các nút trong B , với $G[V \setminus B]$ là đồ thị con của G sau khi loại bỏ B .

Phát biểu bài toán IMIN: Cho một mạng lưới $G = (V, E)$, tập seed S , và ngân sách k . Bài toán IMIN tìm kiếm một tập chặn B^* chứa tối đa k nút sao cho kỳ vọng số nút được bảo vệ là lớn nhất:

$$B^* = \arg \max_{B \subseteq (V \setminus S), |B| \leq k} D_S(B)$$

Bài toán IMIN đã được chứng minh là NP-khó và APX-khó [?]. Hơn nữa, hàm mục tiêu $D_S(\cdot)$ là một hàm đơn điệu không giảm nhưng không có tính chất submodular. Do thiếu hụt tính submodular, việc áp dụng trực tiếp giải thuật tham lam không thể đem lại tỷ lệ xấp xỉ $(1 - 1/e)$ [?].

2.2 Tổng quan các đề xuất hiện có

Khung thuật toán Tham lam Cách tiếp cận nguyên thủy nhất [?] là bắt đầu với tập $B = \emptyset$ và lặp lại k lần. Tại mỗi bước, thuật toán chọn nút mang lại lợi ích cận biên lớn nhất để thêm vào B . Tuy nhiên, việc tính toán kỳ vọng lan truyền là một bài toán #P-khó [?]. Việc sử dụng mô phỏng Monte-Carlo để xấp xỉ giá trị này đòi hỏi chi phí tính toán khổng lồ.

Tham lam dựa trên cây dominator Thay vì tính toán dựa trên mô phỏng ngẫu nhiên, Xie và cộng sự [?] tối ưu hóa quá trình ước lượng lợi ích cận biên dựa trên cấu trúc cây dominator. Trên một realization cụ thể, một nút u được gọi là dominator của v nếu mọi đường đi từ tập seed S tới v đều phải đi qua u . Họ chứng minh rằng số nút được bảo vệ bởi u chính bằng kích thước của cây con có gốc tại u trên Cây chi phối. Dựa trên lý thuyết này, họ đề xuất hai thuật toán: AdvancedGreedy (AG) và GreedyReplace (GR). Mặc dù phương pháp dùng cây dominator có tốc độ nhanh hơn Monte-Carlo, các phương pháp này tồn tại hai hạn chế chí mạng:

- **Sự kém hiệu quả do phải đánh giá lại:** Mỗi khi thuật toán AG hoặc GR chọn xong một nút chặn và đưa vào B , cấu trúc đồ thị thay đổi hoàn toàn. Do đó, thuật toán không thể tái sử dụng các realizations và cây dominator đã được sinh ra ở vòng trước. Để chọn nút thứ hai, hệ thống phải sinh lại toàn bộ tập mẫu từ đầu. Sự lãng phí này khiến thời gian chạy bùng nổ không thể kiểm soát khi ngân sách k lớn.
- **Không có đảm bảo tỷ lệ xấp xỉ lý thuyết:** Do hàm mục tiêu của IMIN là non-submodular, độ hiệu quả của thuật toán tham lam truyền thống phụ thuộc vào tỷ lệ Submodular[?]. Phân tích toán học chỉ ra rằng khi quy mô đồ thị n tăng lên, tỷ lệ ψ sẽ tiến vô hạn về 0. Điều này đồng nghĩa với việc kết quả trả về của các thuật toán hiện đại nhất cho bài toán IMIN hoàn toàn không cung cấp bất kỳ một tỷ lệ xấp xỉ có giá trị nào.

Chiến lược Xấp xỉ Sandwich (Sandwich Approximation Strategy): Nhận thức được giới hạn lý thuyết nghiêm trọng do tính non-submodular mang lại, một hướng tiếp cận khác là sử dụng khung Sandwich [?]. Khung này giải quyết bài toán bằng cách kẹp hàm mục tiêu $D_S(B)$ vào giữa một hàm cận dưới (Lower Bound - D_S^L) và một hàm cận trên (Upper Bound - D_S^U). Cả hai hàm cận này đều được thiết kế toán học để đảm bảo tính chất đơn điệu và submodular, thỏa mãn điều kiện $D_S^L(B) \leq D_S(B) \leq D_S^U(B)$ với mọi tập chặn B .

Cụ thể, thuật toán SandIMIN [?] hoạt động theo ba bước chính: 1) Tìm một phương án đạt xấp xỉ α_1 tối đa hóa hàm cận dưới (thông qua thuật toán LSBM). 2) Tìm một phương án đạt xấp xỉ α_2 tối đa hóa hàm cận trên (thông qua thuật toán GSBM). 3) (Tùy chọn) Tìm một phương án dựa theo một phương pháp heuristic đơn giản. Cuối cùng, thuật toán sẽ đánh giá và trả về kết quả tốt nhất trong ba cấu hình trên. Việc tính toán các hàm cận dưới và cận trên này là #P-khó, đòi hỏi phải sử dụng các kỹ thuật lấy mẫu chuyên biệt. Mặc dù phương pháp Sandwich mang lại một bước tiến về mặt lý thuyết, nó vẫn bộc lộ ba điểm yếu cơ bản trong thực tiễn triển khai. Việc bắt buộc hệ thống phải chạy song song hoặc tuần tự ba thuật toán khác nhau (LSBM, GSBM, LHGA) tạo ra gánh nặng tính toán rất lớn, có thể không phù hợp trong một số kịch bản ứng phó sự cố an toàn thông tin đòi hỏi thời gian phản hồi không được quá lâu.

Để lấp đầy hoàn toàn các khoảng trống này, thay vì cố gắng tính toán dựa trên một khung Sandwich công kênh mà có thể không tốt nếu cả hai hàm chặn đều lỏng, trong bài này chúng tôi đề xuất một hàm chặn

trên duy nhất, chặt chẽ và có tính chất submodular. Bằng cách tối ưu hóa hàm này thông qua việc mở rộng kỹ thuật OPIM-C, phương pháp của chúng tôi chỉ yêu cầu chạy một thuật toán đơn lẻ với quy trình lấy mẫu tối giản, đạt được tỷ lệ xấp xỉ $(1 - 1/e - \epsilon)$ một cách trực tiếp với thời gian tính toán gần như tối ưu.

3 Thuật toán Crucial Path Sampling (CRUP)

Các thuật toán AG, GR hay SandIMIN có một điểm chung về quá trình tiền xử lý. Cụ thể, với một tập seed S cho trước, họ gộp tập này thành một nút nguồn s duy nhất. Khi đó, những nút u có cạnh nối trực tiếp từ một nút nào đó trong S tới nó cũng sẽ có một cạnh (s, u) nối từ s tới nó có xác suất kết nối được tính bằng:

$$p_{(s,u)} = 1 - \prod_{v \in (S \cap N_{in}(u))} (1 - p_{(v,u)})$$

Ngoài ra, ta cũng gọi V_s là tập các nút có đường đi từ s tới nó trong G và $E_s = V_s \times V_s$.

Trong thuật toán SandIMIN, cả 2 thuật toán LSBM và GSBM đều có điểm chung ở các giai đoạn áp dụng thuật toán OPIM-C. Điểm khác biệt lớn nhất chỉ nằm ở quá trình lấy mẫu. Cụ thể, thuật toán LSBM tập trung vào những nút có khả năng thống trị các nút được lấy mẫu. Khi đó, tập các nút này có thể được tìm thấy hiệu quả dựa trên thuật toán dựng cây dominator đã đề cập ở trên. Thuật toán GSBM nói lỏng điều kiện cứu nút được chọn mà chỉ cần tìm những nút nằm trên đường đi từ nguồn tới nút được chọn. Thuật toán của chúng tôi cũng cần một quá trình lấy mẫu để lựa chọn các nút cần quan tâm. Tiếp theo, chúng tôi đưa ra một số định nghĩa cần thiết cho quá trình lấy mẫu của mình.

Definition (Target Reverse Reachable Realization - TaR3): Cho trước một nút v và nguồn S . Từ v , thực hiện duyệt đồ thị ngược về S để tìm các cạnh được kích hoạt. Nếu tồn tại đường đi từ S tới v , đồ thị TaR3 của nút v , kí hiệu là $TaR3(v)$ là đồ thị chứa các nút và cạnh nằm trên đường đi từ S tới v . Ngược lại, đồ thị $TaR3(v)$ là một đồ thị chứa tập nút và cạnh rỗng.

Cho trước một tập nút chặn B . Xét đồ thị $TaR3(v)$ của một nút v cho trước. Để nắm bắt các luồng lan truyền thông tin độc lập và triệt tiêu chu trình, ta tiến hành trích xuất một đồ thị con có hướng không chu trình (Directed Acyclic Graph - DAG) từ $TaR3(v)$. Cụ thể, quá trình này lặp lại việc tìm tập các đường đi ngắn nhất từ S tới v trên phần đồ thị còn lại, đưa chúng vào đồ thị con DAG, sau đó xóa bỏ toàn bộ các đỉnh trung gian của các đường đi này khỏi đồ thị làm việc. Quá trình dừng lại khi không còn đường đi nào tới v . Chi tiết thuật toán được trình bày dưới dạng mã giả của thuật toán 1.

Gọi $\mathcal{P}(TaR3(v))$ là tập hợp tất cả các đường đi từ S tới v tồn tại trên đồ thị con DAG vừa được xây dựng. Nhờ đặc tính không chu trình và tính trích xuất độc lập đỉnh, mọi đường đi hợp lệ trong $\mathcal{P}(TaR3(v))$ đều đại diện cho một luồng lan truyền tiến và tuyệt đối không chứa vòng lặp. Giả sử mọi đường đi trong $\mathcal{P}(TaR3(v))$ có vai trò bình đẳng, ta lấy mẫu một đường đi $\rho \in \mathcal{P}(TaR3(v))$ ngẫu nhiên với phân phối đều.

Khi đó, kỳ vọng số nút cứu được khỏi S sau khi chặn tập nút B thỏa mãn:

$$D(B) \leq U(B) = \sum_{v \in V_S} \mathbb{E}_{\rho \sim \mathcal{P}(TaR3(v))} [V(\rho) \cap B \neq \emptyset]$$

Trong đó, V_S là tập các nút có thể bị ảnh hưởng bởi S trên đồ thị gốc G , $V(\rho)$ là tập các nút nằm trên đường đi ρ .

Definition (Random Target Reverse Reachable Realization - RTaR3): Cho trước một nguồn S . Chọn một nút v ngẫu nhiên, sau đó lấy mẫu đồ thị $TaR3(v)$ ứng với v , ta thu được một đồ thị RTaR3.

Dựa trên định nghĩa về RTaR3, ta thu được một cách biểu diễn khác của hàm $U(\cdot)$. **Định lý:** Hàm chặn trên của $D(B)$ có thể được biểu diễn bởi

$$U(B) = |V_S| \cdot \mathbb{E}_{\rho \sim \mathcal{P}(RTaR3)} [V(\rho) \cap B \neq \emptyset]$$

Khi đó, giả sử ta đã thực hiện được R lần lấy mẫu đường đi, dựa theo định lý, ta biết được công thức ước lượng không chệch lệch cho hàm chặn trên là:

$$U(B) \approx \frac{|V_S|}{R} \cdot \text{số lần } B \text{ xuất hiện trong } R \text{ đường đi đã lấy mẫu}$$

Algorithm 1: CRUP Sampling

Input: Đồ thị ban đầu $G = (V, E)$, tập nút mầm S , nút đích v .

Output: Đường đi ρ phân phối đều từ một mầm bất kỳ trong S tới v trên DAG G_{sub} .

// 1. Xây dựng đồ thị con DAG G_{sub}

```
1 Khởi tạo  $G_{sub} = (V_{sub}, E_{sub})$  rỗng với  $V_{sub} \leftarrow S \cup \{v\}$ ;  
2 Khởi tạo đồ thị làm việc  $G_{temp} \leftarrow G$ ;  
3 while True do  
4   Thực hiện BFS trên  $G_{temp}$  từ  $S$  tới  $v$  để tìm tập  $\mathcal{P}_{SP}$  gồm tất cả các đường đi ngắn nhất;  
5   if  $\mathcal{P}_{SP} == \emptyset$  then  
6     break;  
6     // Dừng khi không còn đường đi  
7    $V_{path} \leftarrow$  tập hợp các nút thuộc  $\mathcal{P}_{SP}$ ;  
8    $E_{path} \leftarrow$  tập hợp các cạnh thuộc  $\mathcal{P}_{SP}$ ;  
8   // Đưa đường đi vào  $G_{sub}$  và xóa nút trung gian khỏi  $G_{temp}$   
9    $V_{sub} \leftarrow V_{sub} \cup V_{path}$ ;  
10   $E_{sub} \leftarrow E_{sub} \cup E_{path}$ ;  
11  Xóa tất cả các nút thuộc  $V_{path} \setminus (S \cup \{v\})$  khỏi đồ thị  $G_{temp}$ ;  
// 2. Forward DP: Đếm số lượng đường đi trên DAG  
12 Sắp xếp các đỉnh trong  $G_{sub}$  theo Thứ tự Tô-pô (Topological Order) thành chuỗi  $T$ ;  
13 Khởi tạo  $dp[u] \leftarrow 0$  cho mọi  $u \in V_{sub}$ ;  
14 for mỗi  $s \in S$  do  
15    $dp[s] \leftarrow 1$ ;  
16 for mỗi nút  $u$  theo thứ tự  $T$  do  
17   for mỗi cạnh  $(u, w) \in E_{sub}$  do  
18      $dp[w] \leftarrow dp[w] + dp[u]$ ;  
// 3. Backward Sampling: Lấy mẫu ngược từ  $v$  về tập  $S$   
19  $\rho \leftarrow \langle v \rangle$ ;  
20  $curr \leftarrow v$ ;  
21 while  $curr \notin S$  do  
22    $Candidates \leftarrow \emptyset$ ;  
22   // Tìm các nút có thể lùi về  
23   for mỗi cạnh  $(u, curr) \in E_{sub}$  do  
24      $\text{Thêm } u \text{ vào } Candidates \text{ với trọng số } W \leftarrow dp[u]$ ;  
25    $Z_{step} \leftarrow \sum_{u \in Candidates} W$ ;  
26   Lấy mẫu nút  $u^* \in Candidates$  với xác suất  $\frac{W}{Z_{step}}$ ;  
27   Chèn  $u^*$  vào đầu đường đi  $\rho$ ;  
28    $curr \leftarrow u^*$ ;  
29 return  $\rho$ ;
```

Thuật toán 1 trình bày chi tiết quy trình lấy mẫu một đường đi trọng yếu (CRUP) phân phối đều từ tập mầm S tới đích v . Mục tiêu của thuật toán không chỉ là tìm một đường đi đơn thuần, mà là giải quyết bài toán cốt lõi: Làm thế nào để lấy mẫu đồng đều trên một không gian đường đi khổng lồ mà không bị mắc kẹt vào vấn đề bùng nổ tổ hợp do các chu trình gây ra. Thuật toán bao gồm ba giai đoạn chính được liên kết chặt chẽ với nhau:

Giai đoạn 1: Xây dựng Đồ thị con DAG độc lập đỉnh (Dòng 1-11) Đây là giai đoạn tiền xử lý quan trọng nhất nhằm triệt tiêu hoàn toàn sự phức tạp tính toán gây ra bởi các chu trình trên đồ thị có hướng G . Thay vì giữ lại toàn bộ đồ thị, thuật toán trích xuất lặp lại các luồng lan truyền độc lập. Cụ thể, vòng lặp **while** (Dòng 4-11) liên tục quét BFS để tìm tập các đường đi ngắn nhất $\mathcal{P}_{SP}$ từ S tới v . Tại Dòng 10: Sau khi chuyển các cạnh và nút của $\mathcal{P}_{SP}$ vào đồ thị con G_{sub} , thuật toán xóa bỏ toàn bộ các đỉnh trung

gian (thuộc V_{path}) khỏi đồ thị G_{temp} . Thao tác này đảm bảo rằng các đường đi được tìm thấy ở các vòng lặp sau sẽ hoàn toàn độc lập đỉnh (vertex-disjoint) với các đường đi ở vòng trước. Nhờ tính chất này, mọi cạnh đi ngược đều bị loại trừ. Kết quả trả về G_{sub} được chứng minh toán học là một đồ thị DAG an toàn tuyệt đối.

Giai đoạn 2: Đếm đường đi bằng Quy hoạch động (Dòng 12-19) Khi đồ thị đã được chứng minh là một DAG, bài toán đếm số lượng đường đi (vốn là #P-khó trên đồ thị tổng quát) được hạ cấp xuống độ phức tạp đa thức thông qua Quy hoạch động. Thuật toán trước tiên áp dụng phép Sắp xếp Tô-pô để tạo ra một trình tự duyệt T sao cho mọi nút cha đều được xử lý trước nút con của nó (Dòng 13). Mảng $dp[u]$ được sử dụng để lưu trữ tổng số lượng đường đi từ tập S đến nút u . Bắt đầu bằng việc khởi tạo $dp[s] = 1$ cho các seed (Dòng 15-17), thuật toán duyệt qua từng nút u theo thứ tự T và truyền số lượng đường đi của nó cho các hàng xóm đầu ra w (Dòng 18-19). Kết thúc giai đoạn này, $dp[v]$ sẽ lưu trữ chính xác tổng số đường đi hợp lệ tồn tại trong không gian G_{sub} .

Giai đoạn 3: Sampling lùi (Dòng 20-30) Giai đoạn cuối cùng thực hiện việc lấy mẫu ngẫu nhiên một đường đi cụ thể ρ dựa trên nền tảng DP đã xây dựng. Thuật toán bắt đầu từ đích v và đi lùi dần về tập mầm S . Tại mỗi nút hiện tại $curr$, hệ thống đánh giá tất cả các nút u có cạnh trở tới $curr$. Xác suất để thuật toán bước lùi về nút u được thiết kế tỷ lệ thuận hoàn toàn với số lượng đường đi $dp[u]$ (Dòng 26). Điều này, trong số các đường đi có thể đan chéo nhau trong G_{sub} , mọi đường đi đều có xác suất được chọn ra là hoàn toàn bằng nhau. Quá trình chen nút vào đầu chuỗi ρ tiếp diễn cho đến khi thuật toán chạm vào bất kỳ một mầm nào thuộc tập S , hoàn thành việc trích xuất một đường đi CRUP hoàn chỉnh.

Algorithm 2: Thuật toán 2

Input: $G(V, E)$, tập seed S , ngân sách k , tham số xấp xỉ (β, δ) , sai số ϵ .

Output: Tập nút cần chặn B_U .

```

1   $ON \leftarrow$  tập các nút được kích hoạt trực tiếp bởi  $S$ ;
2  if  $|ON| \leq k$  then
3    return  $B_U \leftarrow ON$ ;
4   $OPT^L \leftarrow$  hàm chặn dưới của  $U(B_U^o)$ ;
5   $\theta_{\max} \leftarrow \frac{2|V_S| \left( (1-1/e) \sqrt{\ln \frac{6}{\delta}} + \sqrt{(1-1/e) \left( \ln(|V_S| - |S|) + \ln \frac{6}{\delta} \right)} \right)}{\epsilon^2 OPT^L}$ ;
6   $\theta_0 \leftarrow \theta_{\max} \cdot \epsilon^2 OPT^L / |V_S|$ ;
7   $i_{\max} \leftarrow \lceil \log_2 \frac{\theta_{\max}}{\theta_0} \rceil$ ;
8  Sinh 2 dãy đường đi  $\mathbb{P}_1, \mathbb{P}_2$  gồm  $\theta_0$  phần tử theo thuật toán 1;
9   $a_1 \leftarrow \ln \frac{3i_{\max}}{\delta}$ ,  $a_2 \leftarrow \ln \frac{3i_{\max}}{\delta}$ ;
10 for  $i \leftarrow 1$  to  $i_{\max}$  do
11    $B_U \leftarrow$  Max-Coverage( $\mathbb{P}_1, k$ );
12   // Cập nhật cận dưới
13    $\sigma^L(B_U) \leftarrow \left( \left( \sqrt{\text{Cov}_{\mathbb{P}_2}(B_U) + \frac{2a_1}{9}} - \sqrt{\frac{a_1}{2}} \right)^2 - \frac{a_1}{18} \right) \cdot \frac{|V_S|}{|\mathbb{P}_2|}$ ;
14   // Tính Covarage cận trên
15    $\text{Cov}_{\mathbb{P}_1}^u(B_U^o) \leftarrow \min_{0 \leq j \leq k} \left( \text{Cov}_{\mathbb{P}_1}(B_j) + \sum_{v \in \max MC(B_j, k)} \text{Cov}_{\mathbb{P}_1}(v | B_j) \right)$ ;
16   // Cập nhật cận trên
17    $\sigma^U(B_U^o) \leftarrow \left( \sqrt{\text{Cov}_{\mathbb{P}_1}^u(B_U^o) + \frac{a_2}{2}} + \sqrt{\frac{a_2}{2}} \right)^2 \cdot \frac{|V_S|}{|\mathbb{P}_1|}$ ;
18   if  $\sigma^L(B_U) / \sigma^U(B_U^o) \geq 1 - 1/e - \epsilon$  or  $i = i_{\max}$  then
19     return  $B_U$ ;
20   Nhân đôi kích thước của  $\mathbb{P}_1$  và  $\mathbb{P}_2$  bằng các dãy đường đi mới;

```

Với thuật toán CRUP, ta hoàn toàn có thể áp dụng trực tiếp vào khung thuật toán OPIM-C như thuật toán SandIMIN mà không cần thay đổi nhiều về mặt chứng minh lý thuyết.

4 Thực nghiệm

A Cây Dominator và thuật toán AdvacnedGreedy

Trong bài toán Giảm thiểu Lan truyền (IMIN), việc đánh giá chính xác số lượng nút được bảo vệ khi chặn một nút cụ thể là một thách thức lớn. Xie và cộng sự đã đề xuất một phương pháp tiếp cận thanh lịch dựa trên lý thuyết cây dominator. Xét một đồ thị $G(V, E)$ có một nút nguồn s . Một số khái niệm cơ bản về cây dominator bao gồm:

- **Dominator:** Một nút u được gọi là kẻ chi phối của nút v nếu và chỉ nếu mọi đường đi từ s đến v trên g đều phải đi qua u .
- **Immediate Dominator:** Nút u được gọi là thống trị trực tiếp v , ký hiệu $idom(v) = u$, nếu u là dominator của v và mọi nút chi phối khác của v đều chi phối u (nghĩa là u nằm gần v nhất trên các đường đi từ s).
- **Dominator Tree:** Cây dominator của g là một cây có gốc tại s , được tạo thành bởi tập các cạnh có hướng $\langle idom(v), v \rangle$ với mọi v có thể tiếp cận được từ s .

Thuật toán Lengauer-Tarjan được sử dụng để xây dựng Cây chi phối trong thời gian gần như tuyến tính $\mathcal{O}(|E| \log |V|)$, giúp cho việc đánh giá lợi ích của mọi nút ứng cử viên trở nên cực kỳ hiệu quả trên mỗi mẫu đồ thị. Dựa trên lý thuyết Cây chi phối, thuật toán AG được thiết kế để tuần tự chọn ra b nút chặn (ngân sách b). Ở mỗi vòng lặp, thuật toán gọi hàm DecreaseESComputation để ước lượng lợi ích cận biên cho tất cả các nút, sau đó chọn nút có $\Delta[u]$ lớn nhất.

Algorithm 3: DecreaseESComputation(G, s, θ)

Input: Một đồ thị G , nguồn s và số lượng đồ thị mẫu θ .

Output: $\Delta[u]$ cho mỗi $u \in V(G) \setminus \{s\}$, là mức độ giảm kỳ vọng lan truyền khi chặn u .

```
1  $\Delta[\cdot] \leftarrow 0$ ;  
2 for  $i \leftarrow 1$  to  $\theta$  do  
3   Sinh một đồ thị mẫu  $g$  từ  $G$  ;  
4    $DT \leftarrow$  cây chi phối của  $g$  được xây dựng bằng thuật toán Lengauer-Tarjan;  
5    $c[\cdot] \leftarrow$  kích thước của cây con trong cây  $DT$  khi mỗi đỉnh làm gốc;  
6   for mỗi  $u \in V(g) \setminus \{s\}$  do  
7      $\Delta[u] \leftarrow \Delta[u] + c[u]/\theta$ ;  
8 return  $\Delta[\cdot]$ ;
```

Algorithm 4: AdvancedGreedy(G, s, b, θ)

Input: Một đồ thị G , nguồn s , ngân sách b và số lượng đồ thị mẫu θ .

Output: Tập nút chặn B .

```
1  $B \leftarrow \emptyset$ ;  
2 for  $i \leftarrow 1$  to  $b$  do  
3    $\Delta[\cdot] \leftarrow$  DecreaseESComputation( $G[V \setminus B], s, \theta$ );  
4    $x \leftarrow -1$ ;  
5   for mỗi  $u \in V(G) \setminus (\{s\} \cup B)$  do  
6     if  $x = -1$  or  $\Delta[u] > \Delta[x]$  then  
7        $x \leftarrow u$ ;  
8    $B \leftarrow B \cup \{x\}$ ;  
9 return  $B$ ;
```

B Sửa lại thuật toán LSBM (SandIMIN)

Các tác giả trong SandIMIN đã mắc một sai lầm trong chứng minh một bổ đề được dùng để đánh giá các cận dưới, cận trên cho khung OPIM-C. Điều này dẫn đến thuật toán bị sai lầm về mặt lý thuyết (dù vẫn có

thể đạt hiệu suất ổn). Cụ thể, tại chứng minh bổ đề 5.1, các tác giả đã đánh giá rằng $p = \frac{\text{Cov}_{C_i^s}(B)}{\mathbb{E}[I_G(s)]} \in [0, 1]$, trong khi đại lượng này có thể trở nên lớn hơn 1. Điều này dẫn đến cả bổ đề về đánh giá cận dưới và cận trên của khung OPIM-C đều bị sai. Giải pháp đưa ra của chúng tôi là thay vì lấy mẫu một nhóm nút mỗi lần như Local Sampling, ta chỉ lấy mẫu 1 nút như thông thường. Điều này không làm thay đổi hiệu suất, độ phức tạp lý thuyết của thuật toán cũng như không cần phải thay đổi khung thuật toán OPIM-C.

Let $C_1^s, C_2^s, \dots, C_\theta^s$ denote θ random CP sequences in $\mathbb{C}^s$, $\frac{\text{Cov}_{C_i^s}(B)}{\mathbb{E}[I_G(s)]} = x_i$ and $\frac{D_s^L(B)}{\mathbb{E}[I_G(s)]} = p$. Since each C_i^s is generated from a random realization, for any $i \in [1, \theta]$, we have

$$\mathbb{E}[x_i \mid x_1, x_2, \dots, x_{i-1}] = \mathbb{E}[x_i] = p.$$

Let $M_i = \sum_{j=1}^i (x_j - p)$, thus $\mathbb{E}[M_i] = \sum_{j=1}^i \mathbb{E}[(x_j - p)] = 0$, and

$$\begin{aligned} \mathbb{E}[M_i \mid M_1, M_2, \dots, M_{i-1}] &= \mathbb{E}[M_{i-1} + (x_i - p) \mid M_1, M_2, \dots, M_{i-1}] \\ &= M_{i-1} + \mathbb{E}[x_i] - p = M_{i-1}. \end{aligned}$$

Therefore, $M_1, M_2, \dots, M_\theta$ is a martingale. Since $x_i, p \in [0, 1]$, we have $M_1 \leq 1$ and $M_k - M_{k-1} \leq 1$ for any $k \in [2, \theta]$. In addition,

$$\text{Var}[M_1] + \sum_{k=2}^{\theta} \text{Var}[M_k \mid M_1, M_2, \dots, M_{k-1}] = \sum_{k=1}^{\theta} \text{Var}[x_k].$$

Hình 1: Lỗi sai trong chứng minh lý thuyết của LSBM

Algorithm 5: Fixed Local Sampling

Input: Đồ thị $G = (V, E)$, tập nút mầm S , và nút đích v .

Output: Tập hợp các nút thông trị của v , ký hiệu là $\text{Dom}_\phi(v)$.

- 1 Tạo một realization ϕ từ đồ thị G và tìm tập các nút có thể được s chạm tới $R_\phi(s)$;
 - 2 **if** $v \notin R_\phi(s)$ **then**
 - 3 **return** $\emptyset$;
 - 4 Áp dụng thuật toán Lengauer-Tarjan trên đồ thị con cảm sinh bởi $R_\phi(s)$ để dựng cây dominator;
 - 5 $\text{Dom}_\phi(v) \leftarrow \{v\}$;
 - 6 $\text{curr} \leftarrow v$;
 - 7 **while** $\text{idom}[\text{curr}] \neq s$ **do**
 - 8 $\text{curr} \leftarrow \text{idom}[\text{curr}]$;
 - 9 $\text{Dom}_\phi(v) \leftarrow \text{Dom}_\phi(v) \cup \{\text{curr}\}$;
 - 10 **return** $\text{Dom}_\phi(v)$;
-

Thuật toán LSBM lúc này có thể được thực hiện theo khung thuật toán OPIM-C mà không cần thay đổi các đánh giá nào cả, quá trình lấy mẫu được dựa trên Fixed Local Sampling.